

Player.py

Student worksheet - build the poker player model

Create a file called `player.py`. Work through the steps in order. Each part adds one small piece to the final program. The final version should be able to represent a poker player, store cards and chips, and handle betting.

This worksheet does not assume you have implemented `cards.py` before. You can treat `Card` as a placeholder object that represents a playing card.

Before you start: Use meaningful names and keep the code readable. Test after each section instead of writing the whole file first.

0. Set up the file

At the top of `player.py`, add the imports you will need.

```
from dataclasses import dataclass, field
from .cards import Card
```

Hint: `field(default_factory=list)` creates a fresh empty list for each new `Player` instance

1. create the class `Player`:

Use `@dataclass` above the class. Add the following attributes:

- `name`
- `chips`
- `is_human` (default `False`)
- `hole_cards` (empty list by default)
- `current_bet` (default `0`)
- `folded` (default `False`)

```
@dataclass
class Player:
    name: str
    ...
```

Checkpoint:

```
player = Player('Alice', 1000)
print(player.chips) # expected: 1000
```

2. create the method `reset_for_hand(self)` (in class `Player`):

This method resets the player for a new poker hand:

- clear the hole cards
- reset `current_bet` to 0
- reset `folded` to `False`

3. create the method `receive(self, cards)` (in class `Player`):

This method should add new cards to the player's hand.

Hint: `extend()` adds all items from another iterable, whereas `append()` adds only a single item.

4. create the method `bet(self, amount)`:

This method handles betting chips:

- raise `ValueError` if `amount` is negative
- calculate the actual wager
- subtract the wager from the player's chips
- increase `current_bet` by the wager
- return the wager

```
if amount < 0:
    ...
```

Hint: Use `min(amount, self.chips)` so a player cannot bet more chips than they currently own.

Checkpoint:

```
player = Player('Bob', 100)
player.bet(25)
print(player.chips)          # expected: 75
print(player.current_bet)    # expected: 25
```

5. create the `@property` active:

This property should return `True` if the player is still active. A player is active if:

- they have not folded
- they still have chips

```
@property
def active(self) -> bool:
    ...
```

Common mistakes to check

- Using [] directly instead of field(default_factory=list), which causes all instances to share the same list.
- Forgetting to reset folded in reset_for_hand.
- Using append instead of extend when receiving multiple cards.
- Allowing negative bet amounts.
- Forgetting to return the wager from the bet method.

Optional extension

- Add an all_in() method.
- Track each player's total winnings across multiple hands.
- Add player statistics such as win rate.
- Create a meaningful string representation using __repr__ or __str__.